LABORATÓRIO DE BANCO DE DADOS · UCB 2026/2

Escola de Música

Etapa 2 — operação, interface e desempenho

Group 8 · Prof. Samuel Novais

MySQL 8 · Python + Flask

Demonstração ao vivo

15 minutos

ROTEIRO · 15 MINUTOS

Os momentos que o enunciado pede, em quatro partes

1

Contexto e cadastro

Domínio, o que mudou e o CRUD de um aluno

3 min · Gabriel

2

Consultas e restrições

Consulta parametrizada, relatório por visão e erro tratado

4 min

3

Transação

Sucesso, falha provocada e o banco íntegro nos dois casos

4 min

4

Desempenho e acesso

EXPLAIN antes e depois do índice; o mesmo sistema em dois perfis

3 min

+ 1 minuto de arguição. Cada parte termina com a aplicação e o banco rodando ao vivo.

O DOMÍNIO

Do instrumento à mensalidade, em uma linha



Requisito do enunciado	Onde está no modelo
Especialização	PESSOA → ALUNO / PROFESSOR (total e sobreposta)
Autorrelacionamento	NÍVEL → NÍVEL (pré-requisito)
Entidade fraca	AVALIAÇÃO, identificada pela matrícula
Atributo temporal	HISTÓRICO_SITUAÇÃO da matrícula

DESDE A ETAPA 1

Cada perfil entra no banco com um usuário próprio

2 tabelas novas: USUARIO_APP (senha em hash) e AUDITORIA

3 visões: contato do aluno, ocupação de turma, financeiro

4 papéis no MySQL, com GRANT e REVOKE

3 índices novos e 2 gatilhos



DEMONSTRAÇÃO 1 · AO VIVO

Cadastrar, buscar e alterar um aluno

1. Entrar como **secretaria** / Secretaria@2026
2. Alunos → Novo aluno: **Maria Teste Apresentação**, CPF 11122233344
3. Buscar “Maria” → editar o e-mail → Salvar

PESSOA e ALUNO são gravados na mesma transação.

CONSULTAS E RELATÓRIO

O usuário escolhe o filtro, o banco agrega

Desempenho

Filtros: instrumento e média mínima. Junta 5 fontes, GROUP BY e HAVING AVG(nota).

Inadimplência

Filtros: período e valor mínimo. Junta 3 tabelas, SUM e HAVING SUM(valor).

Relatório

Lê duas visões: vw_relatorio_financeiro e vw_ocupacao_turma.



INJEÇÃO DE SQL

A mesma entrada, dois resultados

```
Busca de aluno: ' OR 1=1 --
```

0

alunos — com parâmetro (nossa aplicação)

O banco procura um nome que começa com esse texto.

47

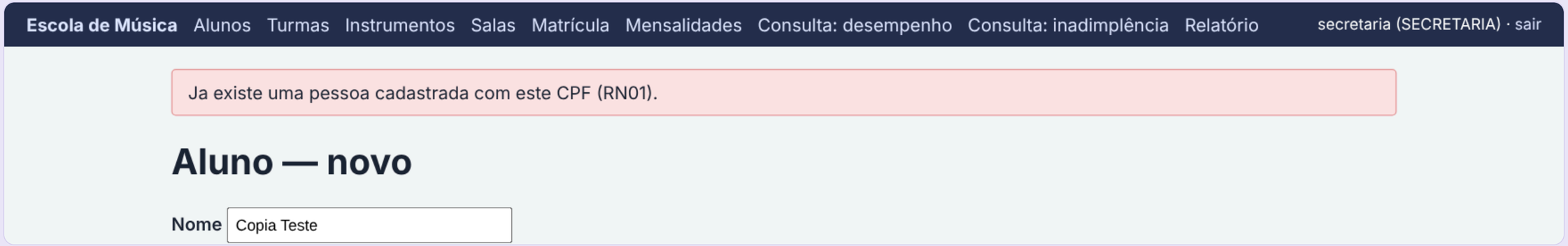
alunos — se o SQL fosse colado como texto

O OR 1=1 vale para todos e o -- comenta o resto.

ERROS TRATADOS

O nome da restrição vira a mensagem da regra

Tentativa	Restrição no banco	O usuário lê
CPF repetido	uq_pessoa_cpf	Já existe uma pessoa com este CPF (RN01)
Excluir aluno com matrícula	fk_matricula_aluno	Não é possível excluir (RN16)
Turma com 50 vagas	ck_turma_vagas	A turma deve ter de 1 a 20 vagas (RN13)



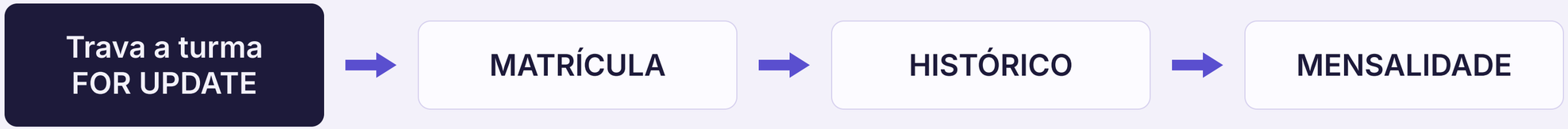
DEMONSTRAÇÃO 2 · AO VIVO

Consulta, relatório e o banco dizendo não

1. Consulta: inadimplência → Consultar
2. Relatório → as duas visões
3. Alunos → buscar **' OR 1=1 --** → nenhum aluno
4. Novo aluno com CPF **68045717847** → recusado
5. Excluir **Arthur Gomes Lima** → recusado

A OPERAÇÃO TRANSACIONAL

Matricular grava três tabelas: ou todas, ou nenhuma



Matrícula sem mensalidade: aluno estudando de graça.

Mensalidade sem matrícula: cobrança de quem não estuda.

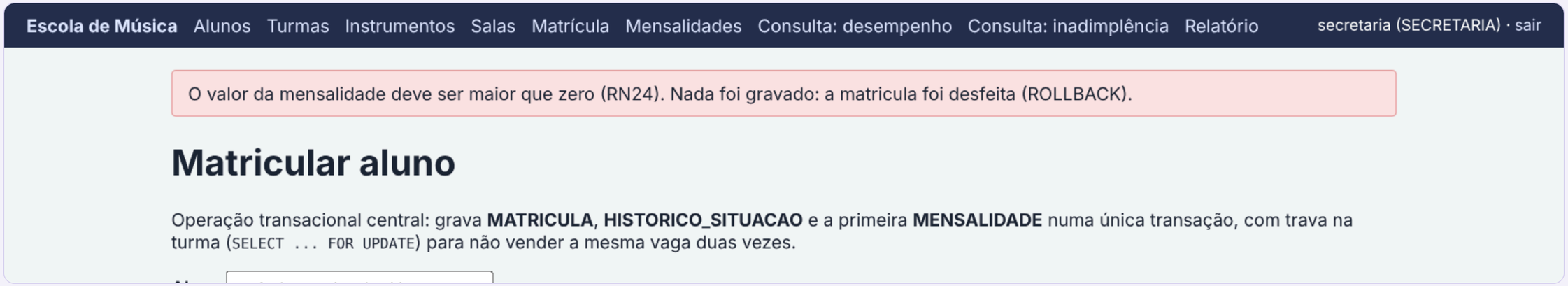
FALHA PROVOCADA

O erro desfaz um comando; o ROLLBACK desfaz o resto

1
matrícula ainda existe, dentro da transação, depois do erro

0
matrículas depois do ROLLBACK

0
históricos órfãos no banco



CONCORRÊNCIA · TURMA COM 1 VAGA

Duas secretárias, a mesma última vaga

SEM TRAVA

9 / 8

As duas leram “7 ocupadas”, as duas gravaram. Turma estourada.

COM SELECT ... FOR UPDATE

8 / 8

A segunda esperou 1,48 s, viu a turma cheia e desistiu.

Isolamento: **REPEATABLE READ**. Trocar o nível não resolve; a trava fica só na turma disputada.

DEMONSTRAÇÃO 3 · AO VIVO

A mesma matrícula com sucesso e com falha

1. **Wagner Rocha Siqueira** · VIO-B1-26C · R$ 320 → COMMIT
2. **Rafael Nogueira Almeida** · VIO-B1-26C · R$ 0 → ROLLBACK
3. Workbench: **sql/09_demo_conferencia.sql** → Wagner 1 · 1 · 1, Rafael 0 · 0 · 0
4. Qualquer aluno · BAT-B2-26B → turma sem vagas

Se pedirem a concorrência: duas conexões no Workbench, SELECT … FOR UPDATE na turma 7.

DESEMPENHO · 50 MIL PESSOAS, 31 MIL MENSALIDADES

O índice vale quando o banco lê menos linhas

Consulta	Índice	Antes	Depois
Busca por nome	(nome)	12,3 ms	0,18 ms
Mensalidade vencida	(data_pagamento, data_vencimento)	6,8 ms	1,4 ms
Relatório mensal	(competencia, data_pagamento, valor)	16,6 ms	8,2 ms

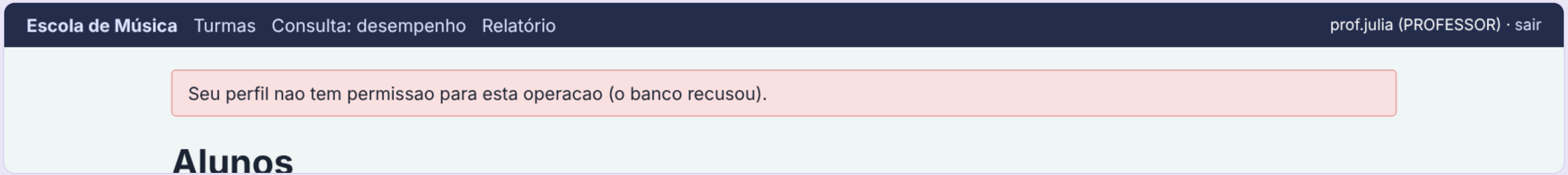
Ordem do composto: igualdade (IS NULL) antes, faixa (<) por último.

Resultado honesto: o relatório ainda lê tudo; o ganho variou de 21% a 51%.

SEGURANÇA E ACESSO

Quem barra o professor é o banco, não a tela

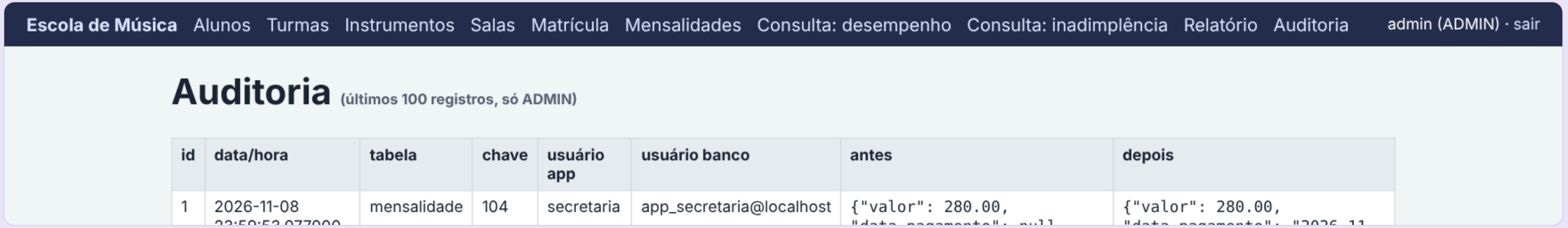
Perfil	Pode	O banco recusa
Admin	Tudo, inclusive a auditoria	Mudar a estrutura
Secretaria	Alunos, turmas, matrícula, mensalidade	Apagar mensalidade (REVOKE)
Professor	Turmas, aulas, notas; alunos pela visão	Ler CPF e mensalidades



Senhas: PBKDF2 com sal aleatório por usuário, 200 mil repetições. Nunca a senha em texto.

AUDITORIA E PROJETO FÍSICO

Quem mudou o dinheiro, quando e de quanto para quanto



2 gatilhos

Mensalidade (é dinheiro) e situação da matrícula (libera vaga).

2 anos

Poucos megabytes. Não precisa particionar.

Backup

Diário às 23h + binlog; 7 diários, 4 semanais, 12 mensais.

DEMONSTRAÇÃO 4 · AO VIVO

EXPLAIN antes e depois, e o mesmo sistema em outro perfil

1. Workbench: rodar **sql/08_demo_explain.sql** → type ALL vira range
2. Sair → entrar como **prof.julia** / Professor@2026
3. Abrir 127.0.0.1:5000/alunos → o banco recusa
4. Consulta: desempenho → Violão, média 6 → funciona

GROUP 8 · ESCOLA DE MÚSICA

Obrigado. Perguntas?

Banco, aplicação, relatório e roteiro estão no repositório. O banco inteiro é recriado do zero com um comando.

RECRIAR TUDO

```
python ferramentas/
preparar_banco.py
```

REPOSITÓRIO

github.com/
gabrielggiani-alt/
escola-de-musica-db